

Student Record Management System

Python Essentials – Evaluated Course Project

Prepared for VITyarthi Course Evaluation

Student: _____

Registration No.: _____

Course: Python Essentials

Academic Project Report

1. Introduction

The Student Record Management System is a menu-driven Python application for managing basic academic records. It provides operations for adding students, searching records, updating marks, calculating results, displaying records, and deleting records.

2. Problem Statement

Manual handling of student records can make searching, updating marks, and calculating results repetitive. The project provides a simple local application that organizes these operations.

3. Objectives

The objectives are to apply Python classes, functions, loops, conditions, dictionaries, file handling, validation, modular programming, and basic testing in a practical application.

4. Functional Requirements

The system provides student addition, search by ID/name, mark updates, total and percentage calculation, PASS/FAIL determination, display of all records, deletion, and JSON-based storage.

5. Non-Functional Requirements

Usability: a clear menu. Reliability: validation and error handling. Maintainability: separate modules. Resource efficiency: local JSON storage with standard Python libraries.

6. System Architecture

The interface is handled by `main.py`. Business operations are handled by `student_manager.py` and `student.py`. `storage.py` manages JSON persistence. `validators.py` validates user input and `reports.py` formats output.

7. Design Decisions & Rationale

A menu-driven command-line interface was selected because it is simple to run, easy to demonstrate, and directly supports the required interaction. JSON was selected for lightweight local storage without external database dependencies.

8. Implementation Details

The `Student` class stores identity, course and marks. `StudentManager` provides record operations. `Validators` prevent empty input and invalid marks. `Storage` serializes records to `students.json`. `Reports` displays formatted results.

9. Screenshots / Results

Before submission, insert screenshots showing the running application: main menu, adding a student, searching a student, updating marks, displaying a result, and displaying all students.

10. Testing Approach

The project includes tests for total calculation, percentage calculation, and PASS/FAIL result logic. Manual testing should also cover invalid marks, duplicate IDs, missing student IDs, search, update, delete, and exit.

11. Challenges Faced

Typical challenges include organizing the project into modules, validating user input, keeping data persistent, and ensuring that updates are reflected in the JSON file.

12. Learnings & Key Takeaways

The project demonstrates how separate Python modules can work together to create a practical application. It also demonstrates classes, data structures, file handling, validation, and testing.

13. Future Enhancements

Possible future improvements include a graphical interface, login and role-based access, a database such as SQLite, export to CSV/PDF, subject management, and more advanced reporting.

14. References

Python documentation and course-provided VITyarthi project instructions. The final submission should also include any additional references actually used by the student.

15. Design Diagrams

System Architecture

User	→	main.py Menu / Interface
main.py	→	student_manager.py Business Logic
student_manager.py	→	student.py Student Model
student_manager.py	→	storage.py JSON Storage
main.py	→	validators.py / reports.py

Workflow Diagram

START	↓
Show Main Menu	↓
Select Operation	↓
Validate Input	↓
Process Student Record	↓
Save / Display Result	↓
Return to Main Menu	■

Use Case Diagram (tabular representation)

Actor	Use Case
User	Add Student
User	Search Student
User	Update Marks
User	Calculate Result
User	Display All Students
User	Delete Student

Sequence Diagram (tabular representation)

User	System	Storage
Select Add Student	Ask for details	
Enter details	Validate input	
	Create Student	Save JSON
Select Display Result	Find student	Read records
	Calculate total/%	
	Show result	

Class / Component Diagram (tabular representation)

Class / Module	Main Responsibility
Student	Stores student data and calculates total, percentage and result
StudentManager	Add, search, update, delete and list records
Storage	Load/save JSON records
Validators	Validate user input
Reports	Display formatted output
Main	Runs the user interface and workflow